

System Design

* Architecture: Internal design details for building the application.

It defines:

- (1) How different components are divided.
- (2) How they interact with each others.
- (3) How data flows through the system
- (4) what technologies & design patterns are used.

(1) Monolithic Architecture: If all the components and functionalities of a project are entangled & combined in a single codebase, then that is a monolithic application.

* All components are tightly coupled & run together as one programs.

Advantages:

- (1) Simple to develop initially
- (2) Easy to test - everything in one place.
- (3) Straight forward deployment.

Disadvantages :

- (1) Hard to scale - We must scale the entire app not parts.
- (2) Difficult to maintain as code grows large.
- (3) Tight coupling : One change can affect other modules.

* Just having folder structure like:

Web-application /
 frontend /
 backend /

does not automatically
means monolithic
architecture.

Architecture is defined by deployment + coupling, not just folders.

- * Are these parts deployed independently?
- * Can they scale independently?
- * Are they tightly coupled or loosely coupled.

If, * frontend deployed on: Vercel / Netify

* Backend deployed on: AWS, Render

* They communicate via REST APIs over the internet.

* We can update frontend without touching

backend.

* Scale backend independently.

Then it is not a monolithic architecture.

It is called a decoupled architecture.

* Difference between Website & Web-Application

(1) Website: Mainly designed to display information to users.

E.x. Wikipedia, GFG, BBC News

(2) Web-Application: A software application that runs in a browser and allows users to perform tasks or operations.

E.x. Gmail, AWS, Google docs.

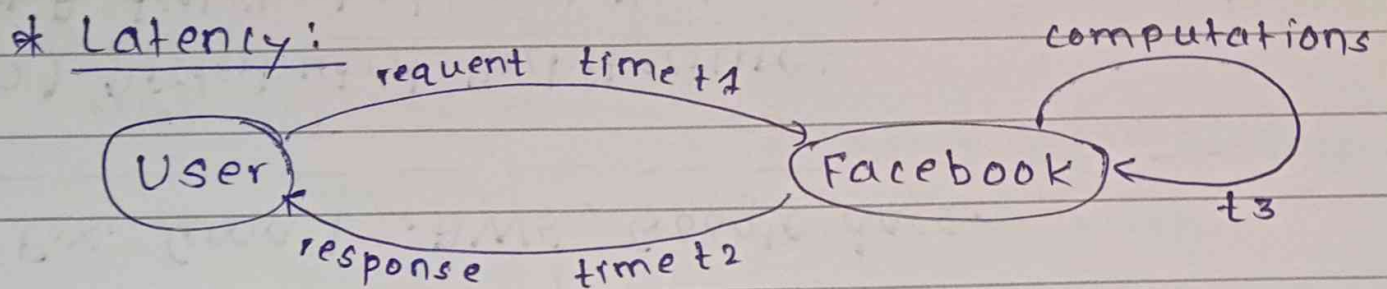
(2) Distributed Architecture:

* A distributed architecture is a system design where: Multiple independent computers/nodes work together over a network to function as a single system.

* Instead of one big system, we have smaller parts running on different machines that communicate with each other.

* Microservices Architecture: A special type of distributed architecture where application is split into small, independent services, each responsible for one business function.

Microservices $\subset$ Distributed Architecture



$$\therefore \text{Latency } \Delta T = \underline{t_1} + \underline{t_2} + \underline{t_3}$$

$$= \text{Network Delay} + \text{computational Delay}$$

Monolithic

$\Delta t = \text{computational delay}$

Distributed

$\Delta t = \text{computational delay} +$

Network delay

Services runs on same machine & communicate by function calls & inmemory operations.		services runs on different machines & communication happens via HTTP / REST APIs.
---	----------	--

* Ways to reduce Latency:

(1) caching: Store frequently accessed data in fast storage (RAM).

- * Avoids hitting slow database or recomputation

- * E.x. Browser cache, Server cache.

(2) Content Delivery Network (CDN):

- * Serves static content from servers closest to user.

- * Reduces geographical distance.

- * Faster delivery of images, videos, static files.

E.x. Cloudflare, Akamai.

(3) Optimize Database queries: Faster data retrieval.

- * Add indexes

- * Avoid unnecessary joins

- * Use pagination.

(4) Load Balancing: Distribute traffic across multiple servers.

E.x. Nginx

(5) Asynchronous Processing: Don't make user wait for long task.

* Faster user response (even if work continues in background).

E.x. Apache Kafka

(6) Reduce Network calls: Minimize no. of API calls.

E.x. Using Graph QL instead of multiple REST calls.

* Throughput: Amount of work a system can handle per unit time.

Measured in: * Requests per second (RPS).

* Transactions per second (TPS)

* Data processed per second (MB/s)

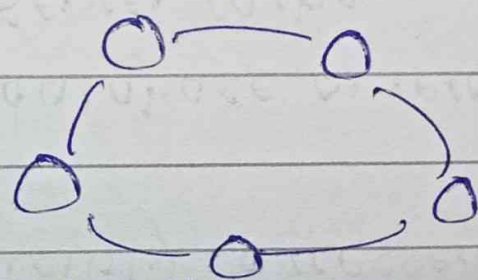
Latency $\rightarrow$ time for one request.

Throughput $\rightarrow$ no. of requests handled overall

Monolithic



Distributed



No limit in resources
load Balancing

causes of Low Throughput:

- (1) Latency
- (2) Protocol overhead
- (3) Congestion.

* Lamport Logical Clock: A concept in distributed systems used to order events in a system where there is no global clock.

* In a distributed machines have different local clocks.

* Clocks may be unsynchronized.

* Instead of real time, we use logical time (numbers) to track the ordering of events.

* Redundancy Vs Replication:

* Redundancy is simply the duplication of nodes or components so that when a node or component fails, the duplicate node is available to service customers.

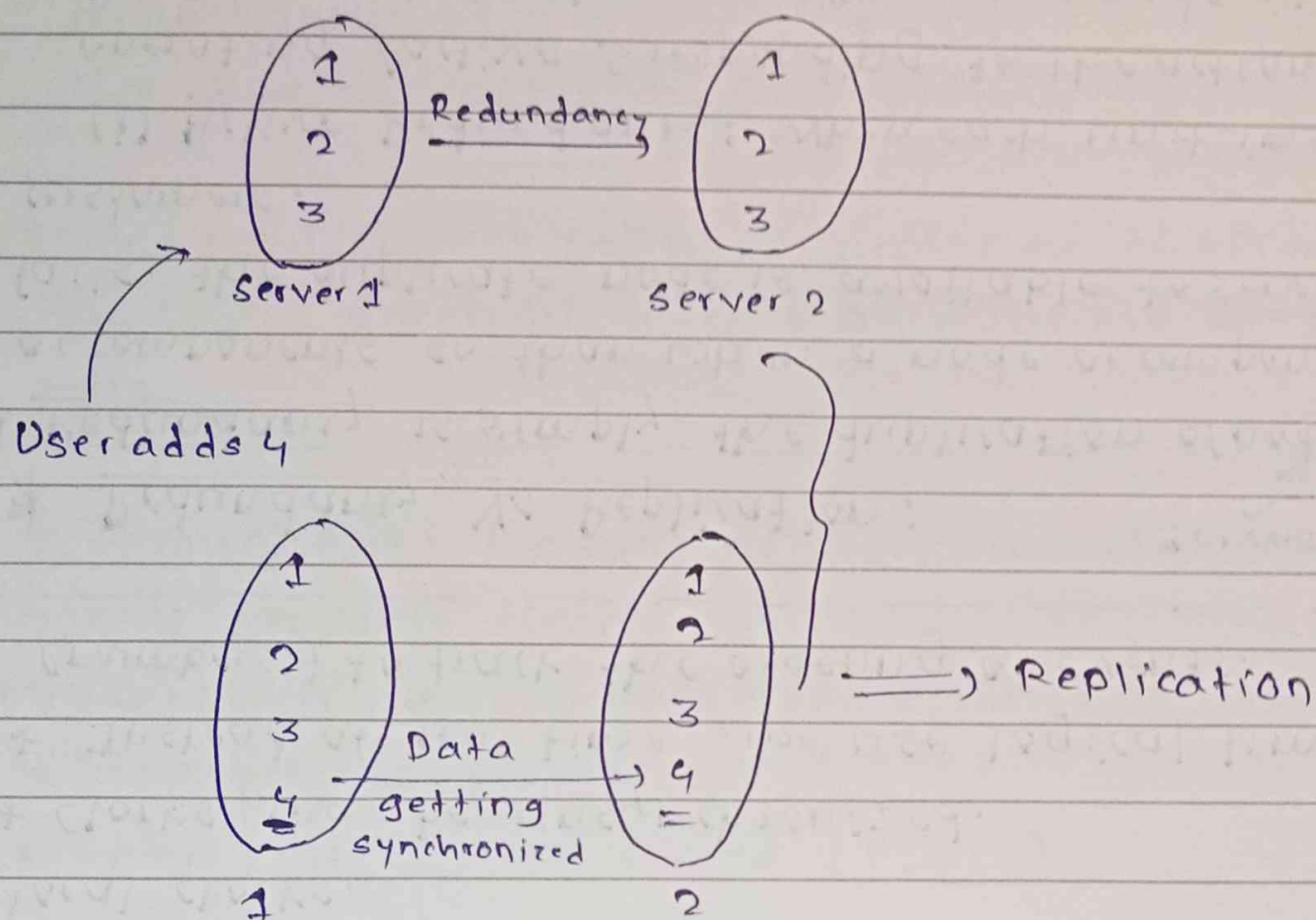
servers



(i) Active Redundancy: When each unit is operating / active & responding to the action.

(ii) Passive Redundancy: When one node is active or operational & other is not operating.

* Replication: Redundancy + Synchronization



* Load Balancer: Load Balancing is the process of efficient distribution of network traffic across all nodes in a distributed system.

Roles of Load Balancer:

- (1) Load distribution equal over every node.
- (2) Health Check: If the node is not operational, the request is passed to another node that

is up and running.

(3) If some servers are added or removed, it automatically manages the traffic with the new servers or with the remaining servers in case of any server removal.

* Load Balancing Algorithms:

- (1) Round Robin: Rotation fashion
- (2) Weighted Round Robin: Similar to round robin, when the servers are of different capacities.
- (3) Least Connection Algorithm: Client requests are distributed to the application server with the least no. of active connections.
- (4) Least Response Time: The request is distributed based on the server which has the ~~the~~ least response time.
- (5) IP Hash: Client IP determines server.
* Same user always goes to same server.
- (6) Random

* Why caching is used:

- * Reduce Latency (Faster responses)
- * Reduce database load.
- * Improve throughput.

* Types of Caching:

(1) Client-Side / Browser caching:

- * Data stored on user device (browser or app)
- * Tools: HTTP headers - Cache-Control
- * Pros → Zero server errors
- * Cons → Tough to update.

(2) Server-Side Caching: Cache stored on backend servers.

- (i) Application Cache: stored in app memory
- (ii) Distributed Cache: shared across multiple servers.

(3) Database Cache: Cache at database layer.

Types of caching based on Eviction:

Eviction: Once the cache is full, any request to add items to the cache might cause existing items to be removed. This is called eviction.

Removing:

(1) Least Recently Used data

(2) Least Frequently used data

(3) TTL (Time to Live) → Data expires after a fixed time.

* Why it is tough to scale RDBMS horizontally:

(1) Vertical Scaling: Increase CPU / RAM of one machine.

(2) Horizontal Scaling: Add more machines (sharding)
It is harder because:

Now your data is split across multiple servers, but queries still expect it to behave like one database.

① ACID Properties: In distributed systems, maintaining strong consistency across nodes is expensive.

E.x. A transaction updates 2 tables stored on different servers. Both must succeed or fail together → requires distributed transactions. (2PC)

In horizontal scaling, the data is primarily split tuple-wise across servers.

② Joins Across Nodes:

```
Select * from Orders o Join Users u  
on o.user-id = u.id;
```

If Orders $\rightarrow$ Server A Users $\rightarrow$ Server B

Then Data must be transferred across network.

Hence, joins becomes expensive & complex.

③ Schema Rigidity: Schema changes must be applied consistently across all shards.

E.x. Adding a new column $\rightarrow$ must update all nodes.

* Why NoSQL Scales Easier:

They: * Avoid joins

* Use flexible schema

* Accept eventual consistency.

* For Payment transactions RDBMS is used
instead of NoSQL: Because of,

(1) Strong ACID Guarantees.

(2) Strong consistency - No stale Data.

RDBMS ensures when a data is read, it is always

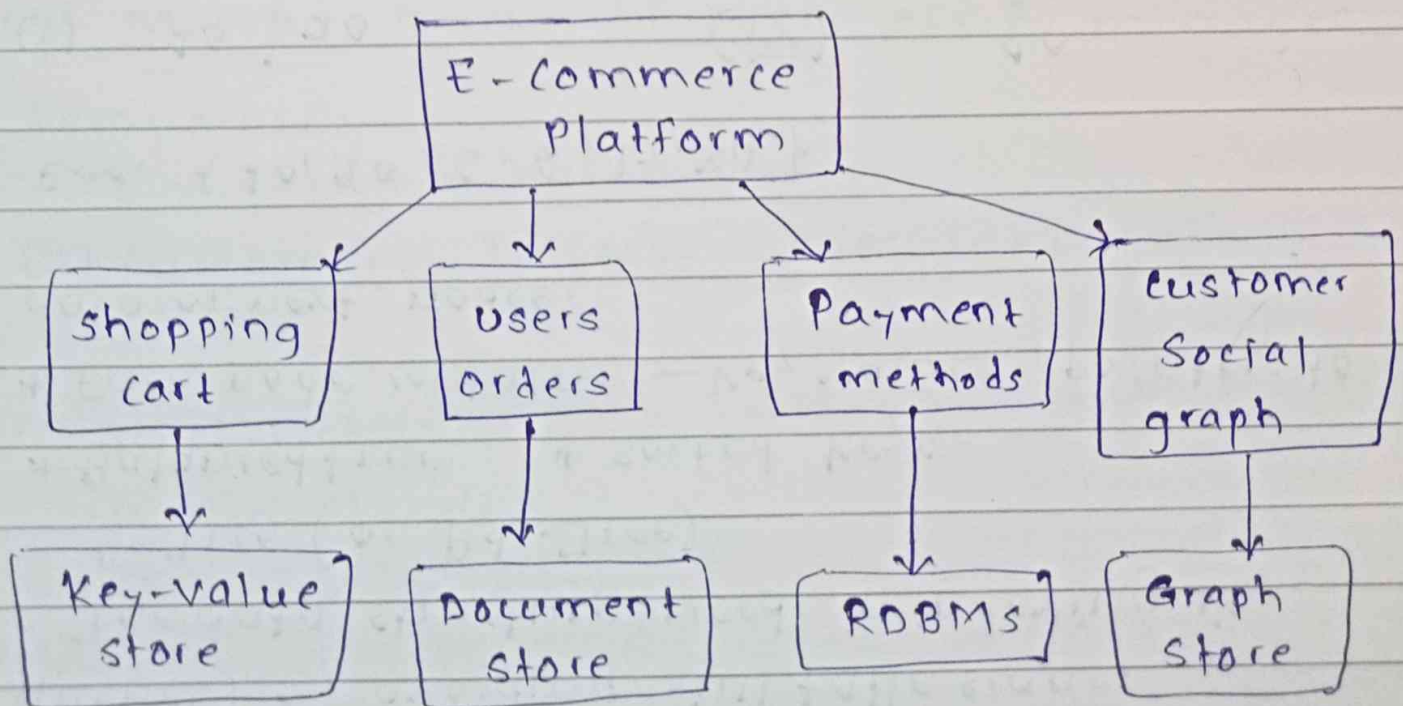
latest & correct.

(3) Transaction support Across multiple rows/tables.

(4) Data integrity via constraints - RDBMS provides built-in constraints (Foreign Key, Primary Key, unique, not null).

(5) Concurrency control (Avoid Race Condition): RDBMS uses locks.

* Polyglot Persistence: Using multiple types of databases / storage systems in a single application, where each is chosen based on its strength.



Indexing: speed up data retrieval (select queries) by avoiding full table scans.

The data structure used in indexing is B-Tree (or B+ -Tree).

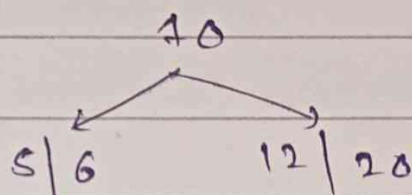
* Balanced tree * sorted keys

* Each node contains — Key values, pointers to data or next nodes.

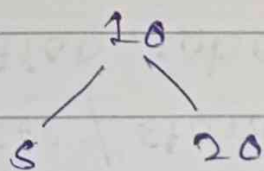
E.x. { 10, 20, 5, 6, 12, 30 }

(i) 10 | 20

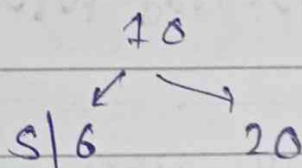
(iv)



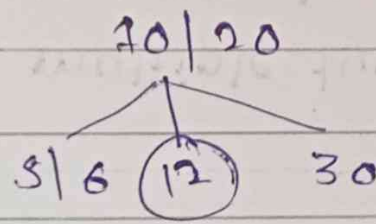
(ii)



(iii)



(v)



12 is between 10

8 20 → go middle child

* Synchronous Communication: (Blocking call)

The sender waits for a response from the receiver before continuing.

Characteristics:

- (1) Blocking nature: Caller must wait
- (2) Immediate Response required: communication happens in real time.
- (3) Tightly coupling: Both systems must be available.

(Non-blocking)

* Asynchronous Communication: Sender does not wait for the receiver's response & continues its work immediately.

Core components:

- (1) Producer: sends message
- (2) Message Queue / Broker: stores messages temporarily.
Ex. Apache Kafka
- (3) Customer: Processes message later.

* Characteristics:

- (1) Non-Blocking
- (2) Loose Coupling
- (3) Event-Driven

* Message-Based Communication:

Different services communicate by sending & receiving messages instead of calling each other directly.

Producer → Message → Queue / Broker → Costumer

* Types of message-based communication:

(1) Point-to-point:

Producer → Queue → One customer

E.x. sending email.

(2) Publish-Subscribe:

Producer → Topic → Multiple Customers

E.x. Notifications, Logging.

* Web Server: A system (software + hardware) that receives HTTP requests from clients (browsers) and return responses like web pages, APIs, images, etc.

* Functionalities:

(1) Handles HTTP Requests

(2) Serves Static Content

(3) Forwards Dynamic Requests: Web servers forwards request to backend.

(4) Manages Connections: Handles thousands of users simultaneously.

Uses techniques like:

- * Threading

- * Even-driven architecture (like Nginx)

(5) Security Features: SSL / TLS

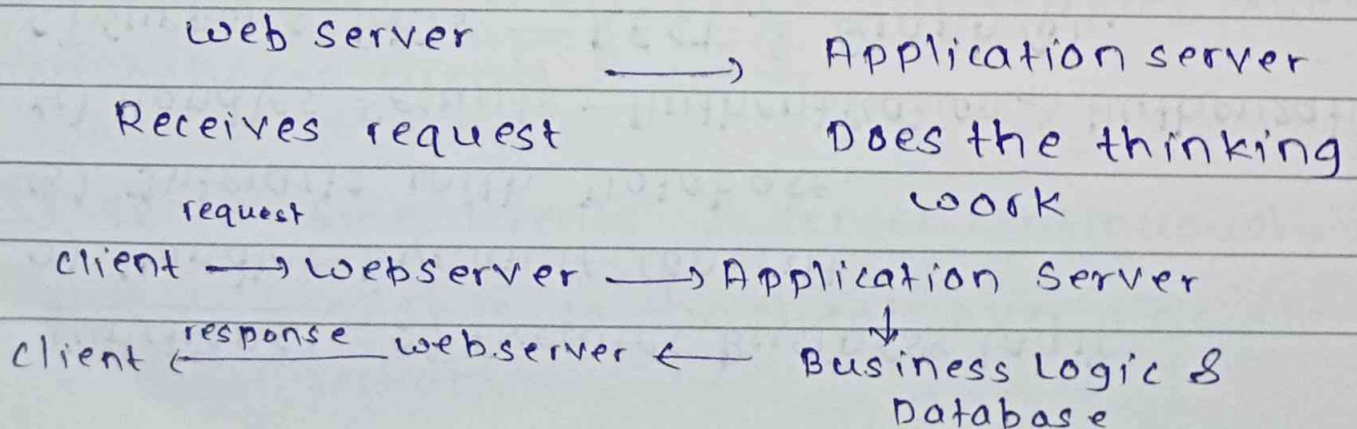
Web
* Types of Servers: E.x. Nginx

(1) Static Web Server → Only serves files

E.x. HTML pages, images

(2) Dynamic Web server → works with backend logic.

* Application Server: A server that executes business logic & processes dynamic requests, interacting with databases & other services to generate responses.



Functions:

- 1) Executes Business logic
- 2) Handles Dynamic Content
- 3) Interacts with Database.
- 4) Handles Security — Authentication & Authorization
- 5) Supports APIs — REST & GraphQL.

E.x. ~~of~~ Node.js, Django, SpringBoot

* Event-Driven Architecture (EDA):

Systems communicate by producing & reacting to events instead of direct requests.

* Core components:

- (1) Event Producer → Generates events.
- (2) Event Broker / Queue → Stores & distributes events. Ex. Kafka, RabbitMQ
- (3) Event Consumers.

Characteristics:

- (1) Loose coupling — Services don't depend on each other directly.
- (2) Asynchronous — No waiting for response
- (3) Reactive — Systems react to events.

* Communication models: Defines how data flows between client & server in real-time systems.

① Push Model: Server pushes data to the client when something happens.

server $\longrightarrow$ client (No request)

E.x. Notifications, Live sport scores,

Requires persistent communication.

② Pull Model (Polling): Client repeatedly asks the server for updates at fixed intervals.

E.x. Refreshing messages every 5 seconds.

Waste resources.

③ Long Polling: Client sends a request, and server holds it open until data is available.

④ WebSockets: A persistent, full-duplex connection between clients & server.

⑤ Server-side Events: A server continuously pushes updates to the client over a single HTTP connection.

Server Side Events (SSE):

* It is one-way communication.

* Features:

- (1) One HTTP requests stays open
∴ There is no repeated requests.
 - 2) server sends updates continuously.
 - 3) If connection drops → browser reconnects automatically.
-

RESTful API: An API that follows the principles of REST (Representational state transfer) to enable communication between client and server over HTTP.

* REST Architectural principles:

- 1) Client-Server Architecture
- 2) Statelessness: Each request must contain all information needed.
* Server does not stores client state.
* Every request is independent.
- 3) Cacheable: Server tells client if response can be cached.

c) Uniform Interface: APIs should follow consistent rules and structure.

Rules: (1) Resource-Based URLs

/users/1, /orders/10

(2) Use HTTP methods:

GET $\rightarrow$ Read, POST $\rightarrow$ Create, PUT $\rightarrow$ Update

DELETE $\rightarrow$ Remove

(3) Standard Representation: JSON / XML

(4) Layered system: client doesn't know backend complexity.

* REST API Format:

Endpoint $\rightarrow$ <https://apiurl.com/review/new>

HTTP method $\rightarrow$ POST

HTTP headers $\rightarrow$ Content-Type: application/json

(key, value Pair)
provides metadata of
request & response

Accept: application/json

Authorization: Bearer < >

Request Body $\rightarrow$ $\{$ "review": $\{$
"title": "Great article",
"rating": 5
 $\}$
 $\}$

Response Body $\rightarrow$ $\{$ "status": "success",
"message": "Review created
successfully"
 $\}$

* Content-Type: application/json

$\rightarrow$ Tells server what type of data we're sending.

* Accept: application/json

$\rightarrow$ Tells us what format we want.

* Common Content-Types:

1. JSON $\rightarrow$ application/json

2. Form Data $\rightarrow$ application/x-www-form-urlencoded

Data is sent like URL query parameters.

E.x. name=Dev&age=22

Use case: HTML forms, Login forms

3. Multipart $\rightarrow$ multipart/form-data
Used to send files + data together.

4. text/plain $\rightarrow$ Raw plain text

5. application/xml $\rightarrow$ Data in XML format

E.x. $\langle$ user $\rangle$
 $\langle$ name $\rangle$ Dev $\langle$ /name $\rangle$
 $\langle$ /user $\rangle$

6. application / octet-stream: Raw binary data
Use Case: File download / upload.

* common Authorization Types:

1. Basic Authentication:

Authorization: Basic base64 (username:password)

2. Bearer Token (JWT)

3. API Key: Public APIs (Google Maps).

4. OAuth 2.0

5. Session-Based Authentication:

Cookie: session_id = abc123

Server Stores session, Client sends session ID.

* Non-RESTful API - SOAP

→ Simple Object Access Protocol

Exchanging structured information between systems using XML over HTTP.



Uses XML only

* Structure is strict

POST /userService HTTP/1.1
- content-Type: text/xml

```
<Envelope>
  <Body>
    <getUser>
      <id>1</id>
    </getUser>
  </Body>
</Envelope>
```

SOAP vs REST:

Features	SOAP	REST
Type	Protocol	Architecture
Format	XML only	JSON/XML
Flexibility	Strict	Flexible
Performance	Slower	Faster
Security	Built-in WS security	External
Complexity	High	Low

* Service - Oriented Architecture (SOA):

An architectural style where application are built as a collection of loosely coupled services that communicates over a network.

Key features: Reusable Services, Loose Coupling

* Microservices: A system design where an application is divided into small, independent services each responsible for a specific function.

Key Features: (1) small & focused services

(2) Independent Deployment

(3) Each service has its own database

(4) Light weight communication - REST APIs

Feature	SOA	Microservices
Size	Large services	Small services
Communication	Central	Direct API based
Protocol	SOAP	REST / JSON
Database	shared	Seperate per service

* SOA: Centralized system with shared services

* Microservices: Fully distributed systems with independent services.

* ERRORS in RESTful APIs: Errors are communicated using HTTP status codes + structured response bodies.

* Client Errors (4xx):

```
{ "status": "error",
  "code": 400
}
```

(1) 400 - Bad Request - Missing fields, invalid json, wrong data format.

2) 401 - Unauthorized - Missing / invalid token.

3) 403 - Forbidden - You are authenticated but not allowed.

4) 404 - Resource does not exist.

5) 405 - Method not allowed - Using POST instead of GET.

6) 409 - conflict - Duplicate email found

7) 422 - Unprocessable Entity - Invalid email format.

2) 5XX - Server Errors :

- 1) 500 - Internal server Error - Generic system (server) failure.
- 2) 502 - Bad Gateway - Server received invalid response from another server.
- 3) 503 - Server Unavailable - Server overloaded or down.
- 4) 504 - Gateway Timeout - Server didn't get response in time.

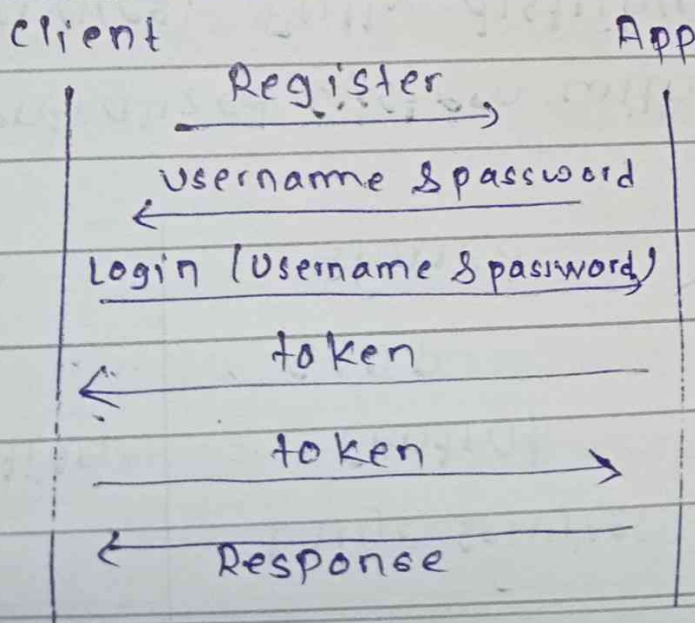
3) Other codes :

200 OK - Success

201 created - Resource created

204 No content - Success but no response body.

Token Based Authentication :



* Proxy Servers: An intermediary server that sits between a client and a destination server, forwarding requests & responses.

Instead of connecting directly →

Client → Proxy → Server

Types of Proxy:

1) Forward Proxy: A forward proxy sits in front of clients and forwards their requests to the internet.

- * Server does not know the real client.

- * It only sees the proxy.

E.x. corporate networks, school internet filtering, VPNs

- * Hides client IP

- * Access blocked websites

- * Content filtering.

2) Reverse Proxy: Sits in front of servers and handles requests from clients.

- * Client does not know actual server

- * E.x. tools → NginX, HAProxy

- * User thinks → He is talking to one server.

- * Actually → Multiple servers behind proxy.

Software development Lifecycle (SDLC):

1) Planning and analysis:

- (i) Identify problem, goals, scope & feasibility.
- (ii) Decide whether the project should be done.
- (iii) Gather user needs and requirements.
- (iv) Create SRS (Software Requirement Specification)

2) Design:

- (i) Plan system architecture, UI, database and workflow.
- (ii) High-level & low-level design.

3) Development / Implementation:

4) Testing: checks for bugs and errors.

5) Deployment: Release the software to users.
can be done in phases or fully.

6) Maintenance: Fix bugs, update features,
improve performance.

* Waterfall Model:

- (1) One of the earliest and simplest approaches in the software development life cycle.
- (2) It follows a linear and sequential process, where each phase must be completed before moving to the next.

Phases of Waterfall model:

1) Requirements Gathering & Analysis:

- * All project requirements are collected & documented. No changes are expected later.
- * Defining feature of a banking app.

2) System Design:

- * Converts requirements into system design,
- * Includes database design, UI design & system structure.

3) Implementation.

4) Testing

5) Deployment

6) Maintenance.

Advantages: * Linear & step-step process.^{by}

* No overlapping phase.

Disadvantages: * changes are difficult once a phase is executed.

* Testing happens late → bound may found late.

* Customer feedback comes very late.

② Agile Model:

(1) Focuses on continuous development, customer feedback and adaptability.

(2) Agile is iterative & incremental, meaning the project is developed in small parts called iterations or sprints.

(3) Each sprint includes:

* Planning * Design * Coding * Testing
* Feedback.

After every sprint, a working product is delivered.

* When to use Agile?

(1) Requirements are uncertain or changing.

(2) complex and large projects.

(3) Need frequent updates and feedback.

* Popular Agile Frameworks:

(1) Scrum (2) Kanban (3) Extreme Programming

(1) Scrum: Lightweight Agile Framework where a team works in small iterations (called sprints) to deliver working software quickly.

There are 3 main roles in Scrum.

(1) Product Owner: * Represents the customer.

* Defines requirements — Product Backlog.

* Defines priority.

(2) Scrum Master: Acts as a coach.

* Ensures the team follows Scrum practices.

(3) Developer Team.

* Scrum Artifacts / Documents:

(1) Product Backlog: List of all features (user stories).

(2) Sprint Backlog: Tasks selected for current Sprint.

(3) Increment: The working product delivered at the end of a sprint.

(2) Kanban: Work is displayed on a Kanban board, which is divided into columns representing stages of work.

E.x. ① To Do ② In Progress

③ Testing ④ Done

Tasks move through different stages of work.

E.x. In a bug-fixing team:

(1) Bugs are added to To-Do

(2) Developers pick and move to -In Progress

(3) After Fixing → Testing

(4) Then → Done

New bugs can be added anytime without waiting.

Kanban Vs Scrum

Feature	Scrum	Kanban
Work style	Sprint-based	Continuous
Roles	Defined	No fixed roles
Changes	Between sprints	Anytime

Real life Kanban board example → Jira

* CI/CD Pipeline:

CI - Continuous Integration: Developers frequently merge code into a shared repository and it is automatically tested.

CD - Continuous Delivery / Deployment: The code is automatically prepared and deployed to production.

* CI/CD Pipeline Stages:

(1) Source Code: Developers push code to repositories like Github or Gitlab.

This triggers the pipeline automatically.

(2) Build stage: code is compiled & dependencies are installed.

Output: Executable app.

(3) Continuous Integration (CI) - Testing stage:

Automated tests runs:

* Unit tests * Integration tests * Code quality

Tools used: Jenkins, CircleCI, Travis CI, checks.

If test fails → pipeline stops.

Else → move forward.

(4) Packaging Stage: Application is packaged into deployable format

E.x. Docker containers using Docker.

(5) Continuous Delivery (CD) - Release Stage

Code is deployed to staging environment for testing.

Tools: Kubernetes, AWS CodePipeline.

(6) Continuous Deployment (Advanced CD):

Fully automated deployment to production.

(7) Monitoring & Feedback.

* Source Control / Version Control: System that helps developers track, manage and store changes in code over time.

* It allows multiple developers to work on the same project without conflicts and makes it easy to back to previous versions if something goes wrong.

Types of Source Control Systems:

① Centralized Version Control System (CVCS):

- * One central server stores all code.
 - * Developers download & upload changes.
- E.x. Subversion (SVN)

② Distributed Version Control: Each developer has a full copy of the repository.

E.x. Git.

* You use Git to track changes in your project on your laptop.

* Then push the code on Github to:

→ Backup the code & collaboration with teams.

* Jenkins: An open-source automation tool used to build, test and deploy software automatically. It is widely used in CI/CD pipelines.

Flow: (1) Developer pushes code to repository (like github).

(2) Jenkins detects the change

(3) Jenkins builds the project.

(4) Runs automated tests (written by developers).

(5) Deploy the application (if tests pass).

This whole process is called Jenkins Pipeline.

* Types of testing in Software Development:

① Functional Testing: checks whether the software behaves according to requirements.

(a) Unit Testing: Tests individual components or functions.

(b) Integration Testing: Tests interaction bet^{wn} modules.

E.x. Login module + Database connection.

(c) System Testing: Testing the complete system as a whole.

② Non-functional Testing: Focuses on performance, security and usability, etc.

(a) Performance Testing: checks speed, responsiveness and stability.

(b) Security Testing: checks for vulnerabilities and threats. E.x. preventing unauthorized access.

(c) Usability Testing: checks user-friendliness of the application.

(d) Compatibility Testing: Tests software on different devices, browsers, OS.

(e) Reliability Testing: Ensures system works without failure over time.

③ Other Important Testing types:

(a) Regression Testing: Ensures new changes don't break existing features.

(b) Smoke Testing: Basic testing to check if the build is stable.

(c) Sanity Testing: Quick check after small changes.

* Deployment Strategies: Defines how new versions of software are released to users with minimal downtime & risk.

① Inplace / Recreate Deployment: Old version is completely removed, then new version is deployed.
Stop old application → Deploy new version → Start application.

Disadvantages: Downtime occurs.

② Rolling Deployment: New version is deployed gradually, replacing old instances one by one.
* No downtime, safer than Recreate.

③ Blue-Green Deployment:

Blue → Current (Live) , Green → New Version
Deploy new version to Green → Test it →
Switch traffic from Blue to Green.

- * Zero downtime
- * Easy switch back
- * Requires double infrastructure.

④ Canary Deployment: Release new version to small group of users first.

⑤ Shadow Deployment: New versions runs in parallel but users don't see it.

- * Real traffic is copied to new system,
- * High resource usage.

* Normalization Examples:

① 2NF: A relationship is in 2NF if

(i) It is in 1NF.

(ii) It has no partial dependency.

Partial dependency: No non-prime attributes should partially depend on part of the candidate key.

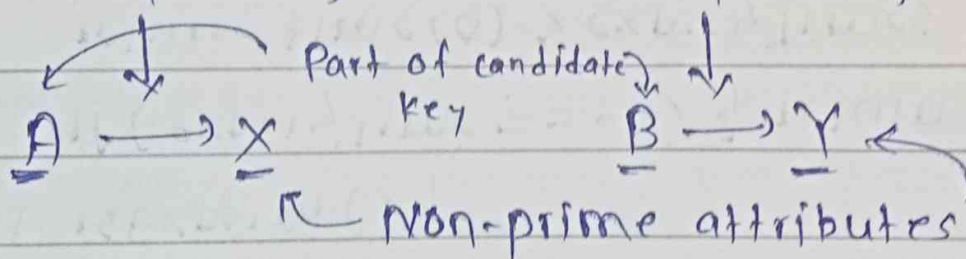
A	B	X	Y
StudentID	CourseID	Name	CourseName
101	C1	XYZ	DBMS
101	C2	XYZ	OS
102	C3	ABC	CNT
102	C2	ABC	OS

candidate key: (StudentID, courseID)

$$AB^+ = \{A, B, X, Y\}$$

$$A^+ = \{A, X\}$$

$$B^+ = \{B, Y\}$$



$\therefore$ There exist partial dependency.

$\therefore$ Relation is not in 2NF.

How to reduce the dependency. Break the table where partial dependency exist:

(A, X), (B, Y), (A, B)

② 3NF: A relation is in 3NF if

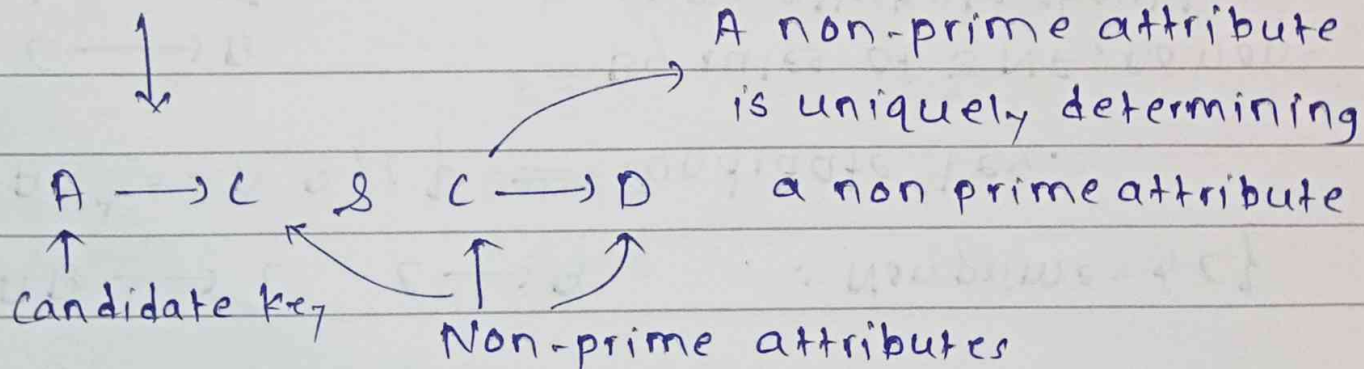
(1) It is already in 2NF.

(2) No transitive dependency exist.

Transitive dependency: No - non-prime attribute should be dependent of another non-prime attribute.

A	B	C	D
EmpID	EmpName	DeptId	DeptName
101	XYZ	D1	HR
102	ABC	D2	IT

candidate key = { EmpID }

$$A^+ = \{A, B, C, D\}$$
$$C^+ = \{C, D\}$$


$\therefore$ This relation is not in 3NF.

To reduce the dependency, Split the table

 $R(A, B, C) \text{ \& } R(C, D)$

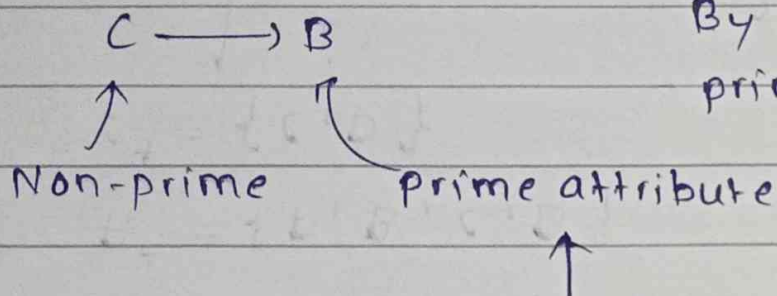
Now there is candidate key

③ BCNF: Removes a special case the 3NF still allows.

A relation is in BCNF if for every functional dependency $X \rightarrow Y$, X must be a super key.

$AB \rightarrow C \quad C \rightarrow B \quad \therefore \text{Non prime} = \{C\}$

$AB^+ = \{A, B, C\} \Rightarrow \text{Candidate Key.}$



By rules of 3NF, no non-prime attribute should depend on other non-prime attribute

But B here is a prime attribute

$\therefore$ Relation is in 3NF.

Now, By rules of BCNF, C must be a super key.

$\therefore$ Relation is not in BCNF.

A Student	B Course	C Instructor
A	DBMS	X
B	DBMS	X
C	OS	Y

FD $\rightarrow (\underline{A, B}) \rightarrow C$, $B \rightarrow C$
 Super Key Not a super Key

∴ Relation is not in BCNF.

To reduce the redundancy, split table:

$R(A, B)$, $R(B, C)$

Lossless

* Example of ~~lossy~~ Decomposition:

S	P	T
Student	Project	Supervisor
Alice	AI	XYZ
Bob	DBMS	ABC
Charlie	DBMS	ABC

Decomposition $R^1(S, P)$, $R^2(P, T)$

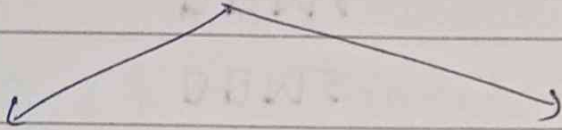
Result after Natural Join on R_1 & R_2

S	P	T
Student	Project	Supervisor
Alice	AI	XYZ
Bob	DBMS	ABC
Charlie	DBMS	ABC

∴ It is a lossless decomposition.

* Loosy Decomposition Example:

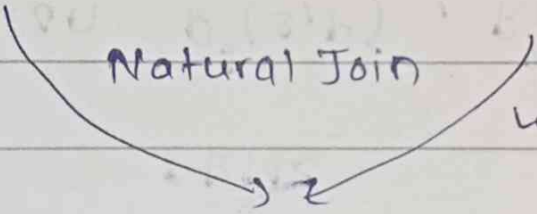
Student	Course	Instructor
Rahul	DBMS	Prof. A
Rahul	OS	Prof. B
ABC	DBMS	<u>Prof. C</u>



Student	Course	Course	Instructor
Rahul	DBMS	DBMS	Prof. A
Rahul	OS	OS	Prof. B
ABC	DBMS	DBMS	Prof. C

Natural Join

Combines rows where course is equal



Student	Course	Instructor
Rahul	DBMS	Prof. A
→ Rahul	DBMS	Prof. C
Rahul	OS	Prof. B
→ ABC	DBMS	Prof. A
ABC	DBMS	Prof. C

A decomposition is lossy if joining table produces:

- (1) Extra (wrong) rows or
- (2) Missing information.

Isolation level: It determines the degree to which the operations in one transaction is isolated from other transaction.

* Anomalies / Violations to isolation level:

① Dirty Read: A transaction reads uncommitted data from another transaction.

E.x. Account A = 10,000

Transaction T1: A $\longrightarrow$ B 7000 $\longrightarrow$ uncommitted

Transaction T2: A $\longrightarrow$ C

T2 reads A's balance as

10,000 $\Rightarrow$ Dirty Read

② Non-Repeatable Read: A transaction reads the same row twice and gets different values.

A = 1000

Transaction T1 $\longrightarrow$ Reads $\longrightarrow$ 1000

Transaction T2 $\longrightarrow$ Update +500 $\longrightarrow$ 1500
 $\longrightarrow$ commits

Transaction T1 again $\longrightarrow$ 1500 $\longleftarrow$ same query
 $\therefore$ This is Non-Repeatable Read. $\longleftarrow$ different result

③ Phantom Read: A transaction re-executes a query and finds new rows (phantoms) added by other transaction.

Transaction T1:

select * from Employees where salary > 25000;
→ Gave 5 rows.

Transaction T2: Inserts new employee with salary 30000.

Transaction T1 again:

select * from employees where salary > 25000
→ Result: 6 rows ⇒ A new row appeared suddenly.
∴ This is Phantom Read.

* To handle these anomalies, DBMS uses transaction isolation levels:

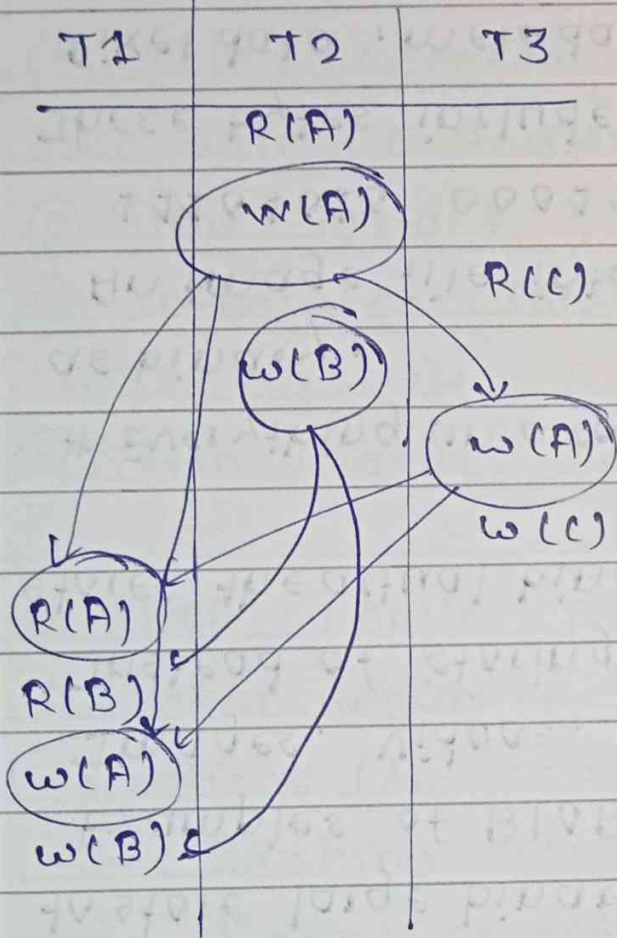
- ① Dirty Read: A transaction can only read committed data.
- ② Non-Repeatable Read: Once a row is read → it gets locked. Other transactions cannot modify it until current transaction ends.
- ③ Phantom Read: Prevents insert/delete affecting your query.

* Conflict Serializable Schedule: A schedule is conflict serializable if we can convert it into serial schedule by swapping non-conflict operations.

* Two operations conflicts if:

- (1) They belongs to different transactions AND
- (2) They access the same data item. AND
- (3) Atleast one of them is write.

E.x. This is a non-serial schedule



To check if this schedule is serializable or not?

* Find conflicts:

W2(A) happens before

W3(A) $\Rightarrow$ T2 $\rightarrow$ T3

W3(A) happens before

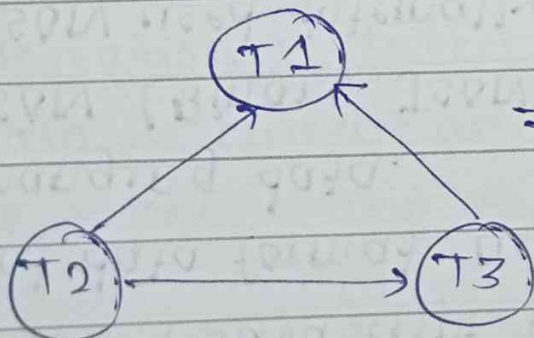
R1(A) & W1(A)

T3 $\rightarrow$ T1

W2(B) happens before

R1(B) & W1(B)

$\therefore$ T2 $\rightarrow$ T1



$\Rightarrow$ No cycle exists

$\therefore$ The schedule is conflict serializable.

Serial order $\rightarrow$ T2 $\rightarrow$ T3 $\rightarrow$ T1

* JSON (JavaScript Object Notation): A text-based data format used for storing and exchanging data.

* BSON (Binary JSON): A binary-encoded format of JSON used internally by MongoDB, for efficient storage & processing.

* BLOB (Binary Large Object): A data type used to store large binary data in database.

Examples of BLOB Data:

Images, Videos, Audio, PDFs

Instead of storing image.jpg, database stores the actual binary content of the image.

* Everything in a computer is ultimately stored as binary.

An image file internally looks like :

11101010 00011010 , , , , ,

These bytes include: File header (JPEG, PNG)

Pixel data, metadata

This entire sequence of bytes is what gets stored in a BLOB.

* Used in SQL databases like MySQL, Oracle

* Data Warehouse: A data warehouse is a system used to store large amounts of historical data from different sources, optimized for analysis and reporting, not day-to-day transactions.

Characteristics:

- ① Subject-oriented: Data is organized by subjects like: sales, customers, products.
- ② Time-Variant: Stores historical data.
- ③ Integrated: Combines data from: Databases, APIs, logs, etc.
- ④ Non-Volatile: Data is mostly read-only.

* OLTP: (Online Transaction Processing):

A system designed to handle day-to-day operations (transactions) quickly and accurately.
E.x. Banking transactions.

Placing an order on an e-commerce site, etc.

Characteristics:

- * Handles many small transactions.
- * Operations: Insert, Update, delete.
- * Data is current and detailed.
- * High speed and concurrency.
- * Tools: MySQL, PostgreSQL.

* OLAP: Online Analytical Processing

A system designed for analytics, reporting and decision-making.

* characteristics: * Handles complex queries.

* Uses historical data.

* Optimized for analysis.

* Tools: Microsoft Redshift, Google BigQuery.

* OLTP: → Systems feed data into a data warehouse

* OLAP: → Systems run on top of that warehouse

* Data Warehouse: → Used to store structured, historical data for analysis & reporting (OLAP)

* Data Center → A physical facility that houses → servers, storage systems and networking equipments.

It is used to run applications, store data and provide computing power.